

并发与锁机制

并发与锁机制（1）

假设某图书销售系统中，事务T1负责统计库存数量大于100的图书种类总数。与此同时，事务T2在T1执行期间插入了一本新书，其库存为150。两个事务的部分调度如下表所示（未使用可串行化隔离级别）：

表：事务执行的调度示意图的一部分

Time	User 1 Transaction	User 2 Transaction
T0	BEGIN	BEGIN
T1	SELECT COUNT(*) FROM books WHERE all-nums > 100; -- 返回结果为3	
T2		INSERT INTO books (book_id, all- nums) VALUES ('B101', 150);
T3		COMMIT
T4	SELECT COUNT(*) FROM books WHERE all-nums > 100; -- 再次查询	
T5	COMMIT	

已知在 T0 时刻，满足 all nums > 100 的图书有 3 种。

- (1) T1在T4时刻第二次执行查询时，返回的结果是多少？
- (2) T1的两次查询结果是否不一致？这反映了哪种并发控制问题？此外，请详细说明导致该问题发生的具体过程。

参考答案：

- (1) 返回 **4**。在 T2 插入并提交后，满足 all-nums > 100 的图书种类从 3 变为 4。
- (2) **不一致**。这是**幻读（Phantom Read）**问题。过程如下：
 - 1. T1 在 T1 时刻统计到 3 条满足条件的记录。
 - 2. T2 在 T2 时刻插入一条新的满足条件记录并提交。
 - 3. T1 在 T4 时刻再次执行相同查询时，看到了新插入的记录，结果变为 4。
 - 4. 在非可串行化隔离级别下，T1 不能阻止其他事务插入满足条件的新行，导致统计结果变化。

并发与锁机制（2）

某在线票务系统支持用户并发购买演唱会门票。系统维护一张余票表 Tickets(event_id, seat_type, available_count)。购票流程如下：用户选择场次和座位类型后，系统读取当前余票数；若余票充足，则将该用户的订单写入订单表 Orders(user_id, event_id, seat_type, quantity)，并更新余票表中的可用数量。

假设将一次购票操作封装为一个事务，并使用以下伪指令表示：

读取余票：R(T)

- 更新余票：W(T,v)，其中 v 为新余票数

写入订单：W(O,q)，其中q为购票数量

则单个事务可表示为： $x = R(T)$; if $x \geq q$ then $W(T, x - q)$; $W(O, q)$

现考虑两个用户同时购买同一场次、同一种座位类型的票，可能出现如下交错执行序列：

$x_1 = R(T); x_2 = R(T); W(T, x_1 - 1); W(O, 1); W(T, x_2 - 1); W(O, 1)$

请回答以下问题：

- (1) 上述并发执行可能导致什么数据一致性问题？请简要说明原因。
- (2) 若采用共享锁 (SLock())、独占锁 (XLock()) 和解锁 (Unlock()) 机制，请设计一个满足两段锁协议 (2PL)、不产生死锁且锁持有时间尽可能短的正确执行序列。

参考答案：

(1) 可能导致**丢失更新/超卖**。两个事务都先读到同一份余票数 x ，各自计算 $x-1$ 后写回，后写入者覆盖前写入者，余票只减少 1，但订单却生成了 2 条，结果就是超卖且余票数不一致。

(2) 一种满足 2PL、无死锁且锁持有时间较短的执行序列（对所有事务统一锁顺序 $T \rightarrow O$ ，避免升级死锁）：

```
T1: XLock(T); R(T); if x1>=q then W(T, x1-q); XLock(O); W(O, q); Unlock(O);
Unlock(T)
T2: XLock(T); R(T); if x2>=q then W(T, x2-q); XLock(O); W(O, q); Unlock(O);
Unlock(T)
```

说明：

- 直接对余票表加 **X 锁**，避免两事务先读后写导致的 $S \rightarrow X$ 升级死锁。
- 所有事务以相同顺序加锁（先 T 后 O ），避免锁顺序反转引发死锁。
- O 的 X 锁只在需要写订单时才申请，缩短锁持有时间。

并发与锁机制（3）

阅读以下说明并回答问题。假设与这两笔业务对应的交易T1和T2与存款关系相关：

- 转账业务--T1(A,B,S)，将S美元从账户A转账到账户B；
- 利息计算业务--T2，为所有活期账户计算利息（即，原始金额为X美元，利息计算后为 $X*1.2$ ）

假设如下表所示，同时调度两笔交易T1和T2，引入(S,U,X)锁（注：兼容性矩阵如下图所示），如果初始 $A = 100$ ， $B = 60$ ， $S = 20$ ，A和B的最终结果是什么？这种并发调度是否正确？

T1(A,B,s)	T2
Read(A)	
A:=A-s	Read(A)
Write(A)	A:=A*1.2
	Write(A)
	Read(B)
Read(B)	B:=B*1.2
B:=B+s	Write(B)
Write(B)	

	Locks already owned by other transactions			
Lock request		S	U	X
	S	Y	Y	N
	U	Y	Y	Y
	X	N	N	N

参考答案（按 18-19.md 第 6 题第 2 问的思路）：

最终结果： $A = 120$ ， $B = 96$ 。

并发调度是否正确： 不正确，不满足可串行化。

简要过程：

1. T1 对 A 申请 **U** 锁并读出 $A=100$ ；T2 也对 A 申请 **U** 锁并读出 $A=100$ （U 与 U 兼容）。
2. T1 将 A 升级为 **X** 锁并写入 $A=100-20=80$ （按题给兼容矩阵可升级）。
3. T2 申请 A 的 **X** 锁被阻塞，等待 T1 释放。
4. T1 对 B 加锁并写入 $B=60+20=80$ ，提交并释放锁。
5. T2 获得 A 的 X 锁，按其旧读值写入 $A=100 \times 1.2=120$ 。
6. T2 读到 $B=80$ ，再写入 $B=80 \times 1.2=96$ 。

该结果既不等于“先转账后计息”(A=96,B=96)，也不等于“先计息后转账”(A=100,B=92)，因此并发调度不正确。